第六章 截交线段拼接计算优化 代码提纲

LinkPoint.py 文件实现：

```
class LinkPoint:
    def __init__(self, pnt3d, digits = 7)
    def __str__(self)
    def toPoint3D(self)
```

LinkSegs_dorder.py 文件实现：

```
def cmp_pntSmaller(lp1, lp2)                # 字典序，比较两个（链接）点大小

class LinkSegs_dorder:
    def __init__(self, segs):
        self.segs = segs                    # 保存线段列表
        self.contours = []                  # 最终得到的轮廓列表 contours
        self.polys = []                     # 用于保存非封闭的轮廓
        self.link()                         # 调用拼接函数
    def createLpList(self)                  # 根据输入线段构建链接点列表
    def findUnusedPnt(self, lpnts):         # 从 lpnts 中找出一个没用过的点
    def link(self):                         # 定义线段字典序拼接核心函数
```

LinkSegs_dlook.py 文件实现：

```
class LinkSegs_dlook:
    def __init__(self, segs):
        self.segs = segs                    # 保存线段列表
        self.contours = []                  # 最终得到的轮廓列表 contours
        self.polys = []                     # 用于保存非封闭的轮廓
        self.link()                         # 调用拼接函数
    def createLpDic(self)                   # 构建字典函数
    def findUnusedPnt(self, dic)            # 从字典中寻找没用过的点
    def findNextPnt(self, p, dic)           # 根据当前点 p 寻找下一个点
    def link(self)                          # 定义链接核心函数 link
```

SliceAlgo.py 文件实现：

```
def linkSegs_dorder(segs)       # 字典序排序线段链接，全局函数
def linkSegs_dlook(segs)        # 字典查询法先端链接，全局函数
```

第六章 截交线段拼接计算优化 代码提纲

MakeMessySeg.py 文件实现

```
def makeMessySegs(circleNum = 10, segNumPerCircle = 100000, radius = 100.0)
                                        #  生成平面上大规模的散乱线段
```

Test_linkSegs.py 文件实现：测试 3 种拼接算法的性能